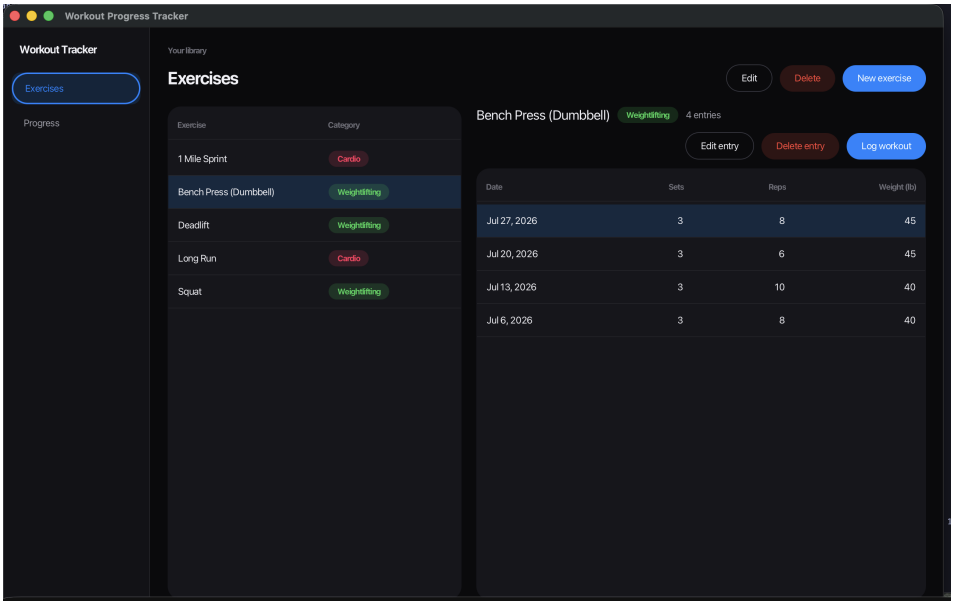
Workout Progress Tracker

A JavaFX desktop application for logging workouts
and tracking progress over time

William Schaffer

Source code: https://github.com/3030Will/a-fitness-tracker

Demonstration video: https://youtu.be/REPLACE-ME



The exercise library, with the log for the selected exercise.

Project Description

Workout Progress Tracker is a desktop application for recording workouts and seeing whether they are getting better. The user defines their own exercises rather than choosing from a fixed list, and each one is either cardio or weightlifting. Sessions are logged against an exercise, and the application summarises what has been recorded and charts how the key measurement has moved.

The two categories are measured differently, and the application takes that seriously rather than flattening it. A weightlifting session records sets, reps and weight. A cardio session records a distance and a time, from which pace is calculated. The form asks only for the fields that apply, the log shows only the columns that apply, and the progress view reports the records that make sense for that kind of exercise.

Problem Statement

People who train regularly usually keep records somewhere — a notebook, a notes app, a spreadsheet. Those work for writing things down and are poor at answering the question the records exist to answer: *am I improving?* Answering it means finding every past entry for one exercise, comparing numbers that are not lined up, and holding the comparison in your head.

A spreadsheet also cannot tell that a row is wrong. Nothing stops a negative weight, a time of 30:70, or a distance recorded against a bench press. Mistakes are found later, if at all, and they quietly corrupt exactly the comparison the records were kept for.

This application addresses both. Every value is checked before it is stored, and the database itself refuses data that would not make sense. Progress is not something the user has to work out — the application keeps the history per exercise, calculates the records, and draws the trend.

Features

Exercises

- Create, rename and delete exercises defined by the user.
- Each exercise is cardio or weightlifting, chosen with a segmented control.
- Names are unique regardless of case, so “Squat” and “squat” cannot both exist.
- Deleting an exercise states how many logged sessions will be deleted with it, and asks before doing so.
- An exercise’s category is fixed once it has entries, since the measurements already recorded depend on it.

Logging workouts

- Log a session against any exercise, dated with a date picker that defaults to today.
- Weightlifting sessions record sets, reps and weight in pounds; bodyweight exercises are recorded at zero pounds.
- Cardio sessions record distance in miles and a time typed as mm:ss or hh:mm:ss.
- Pace in minutes per mile is calculated from distance and time.
- Past entries can be edited or deleted, with confirmation naming the entry being removed.

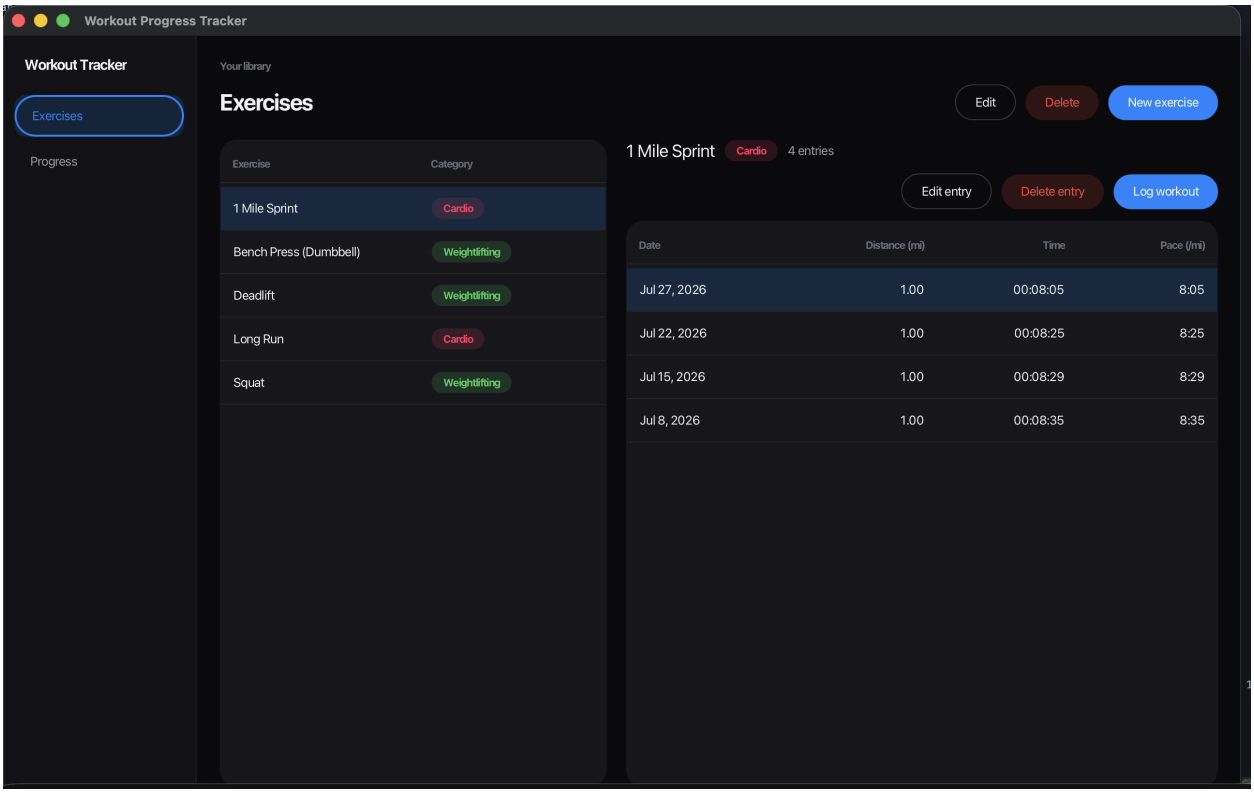
Progress

- Records shown as large readouts: heaviest set, best session volume and sessions logged for weightlifting; longest session, best pace and total distance for cardio.
- A personal-record badge naming the best result and the date it was set.
- A line chart of every session, oldest to newest — weight for weightlifting, pace for cardio.

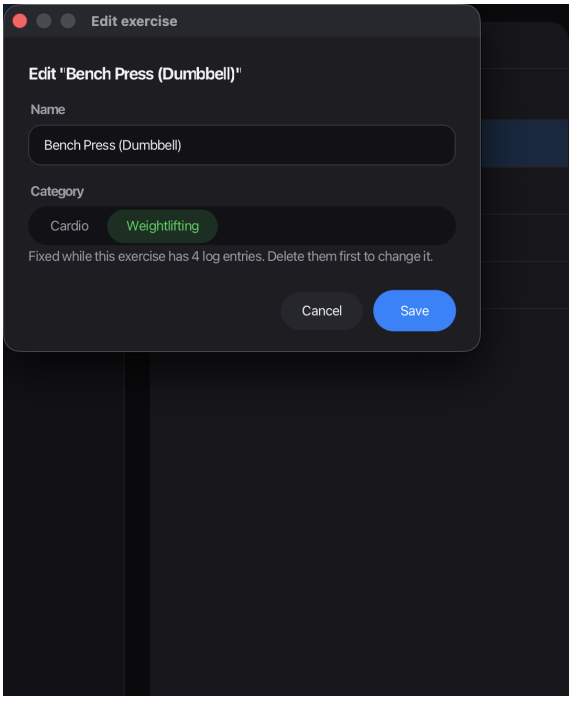
Validation and error handling

- Every problem with a form is reported at once, not one at a time.
- Messages name the field and say what is wrong, and the fields concerned are marked.
- A rejected save leaves the form open with the entered values intact.
- Database failures are reported in plain language rather than as a stack trace.

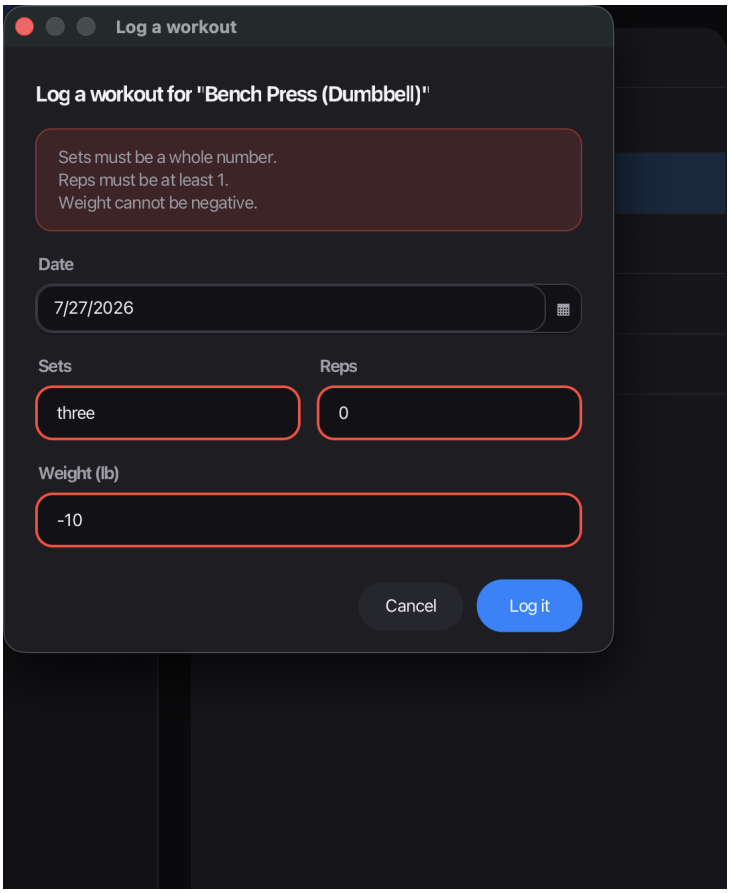
Screenshots



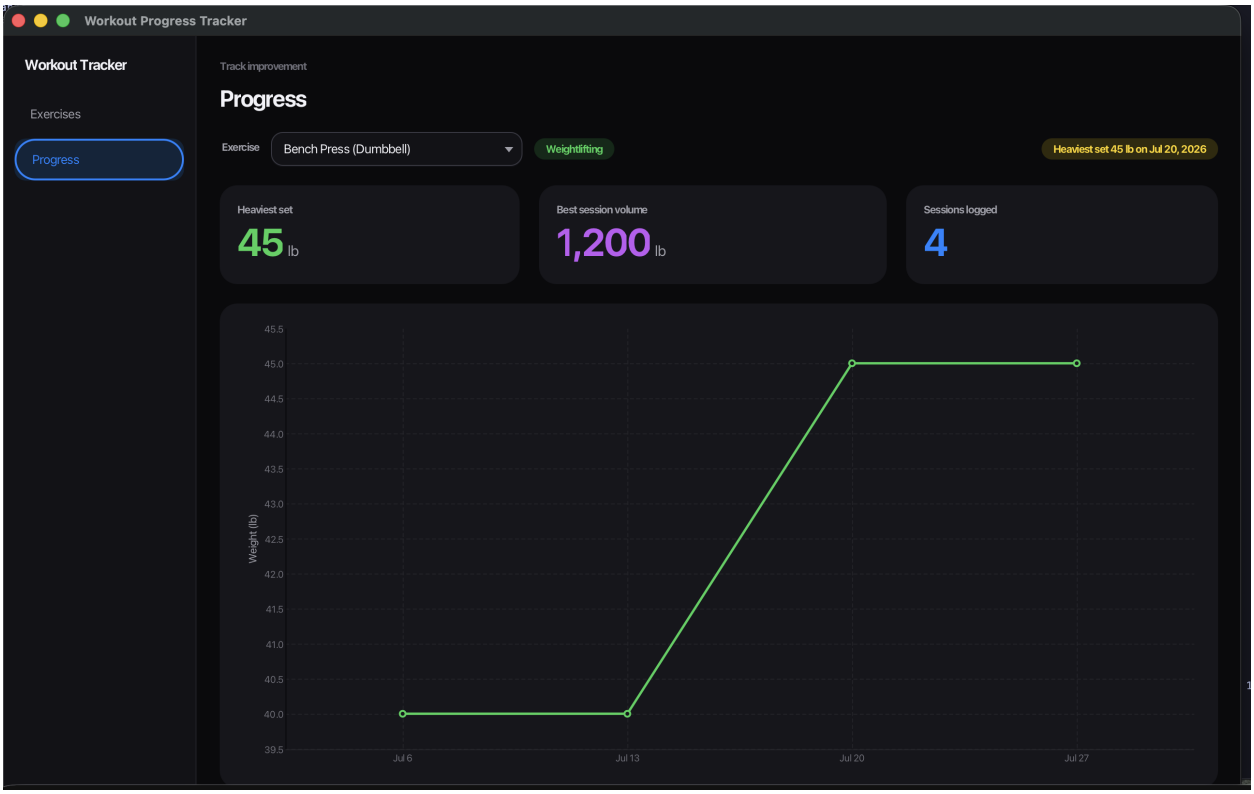
A cardio log. The columns differ from weightlifting, and pace is calculated per session.



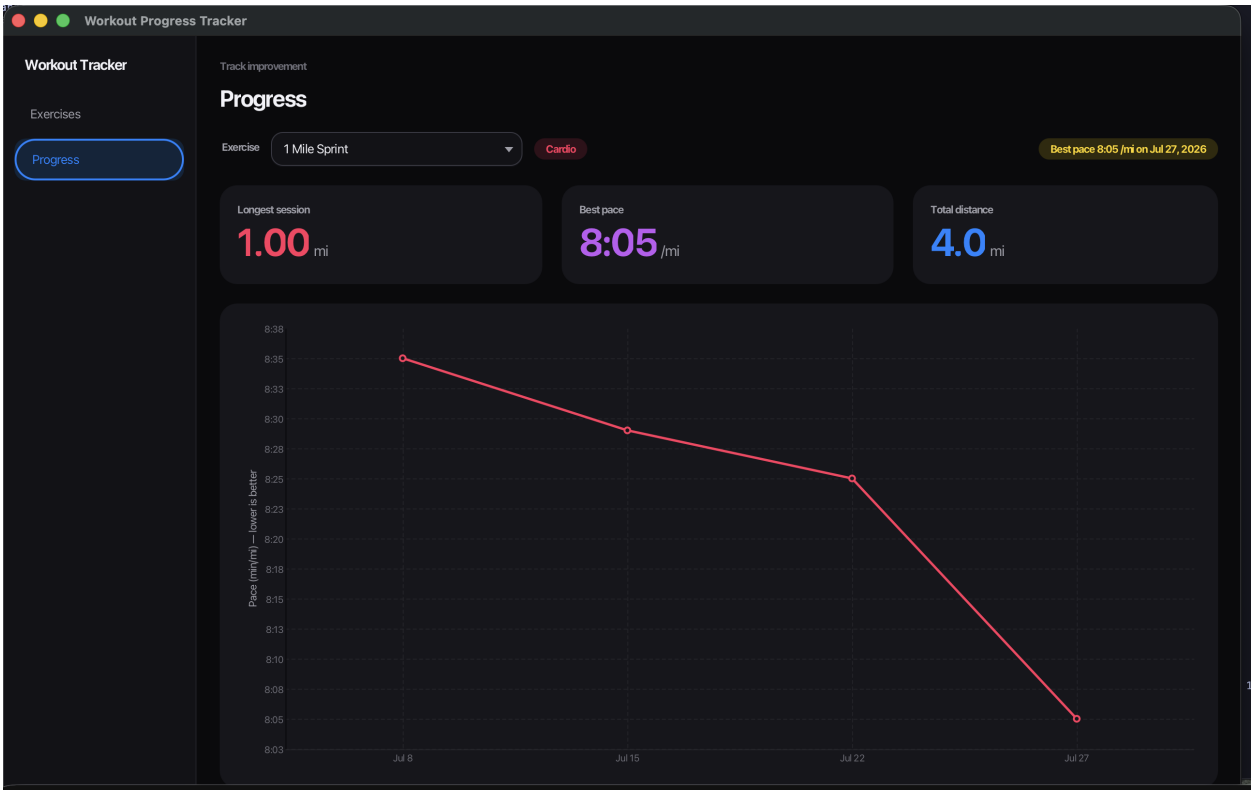
Creating an exercise. The category is chosen with a segmented control.



Three problems reported together, each field marked. The form stays open with the entered values.



Progress for a weightlifting exercise: records, a personal best, and weight over time.



Progress for a cardio exercise. Pace is charted rather than distance, and the axis improves downwards.

Technologies Used

Language	Java 25
User interface	JavaFX 25, laid out in FXML and styled with a single CSS stylesheet
Build	Apache Maven
Database	SQLite, through the org.xerial:sqlite-jdbc driver
Testing	JUnit 5 for logic; TestFX to drive the interface itself

The project is deliberately built without a `module-info.java`. The SQLite driver is only an automatic module, and combining it with JavaFX on the module path produces startup failures that are hard to read. Running on the plain classpath avoids that and costs nothing.

Architecture

The application is layered `ui` → `service` → `dao` → `database`. The interface never contains SQL, and the data access layer never mentions JavaFX.

Cardio and weightlifting sessions share a single `log_entries` table with nullable columns for each kind. They are read back as an abstract `LogEntry` with `CardioEntry` and `LiftEntry` subclasses, and the data access layer chooses the subclass from the parent exercise's category. Each subclass knows how to describe and measure itself, so the interface can display either without asking which it holds.

Rules are enforced where they can be. The database constrains what it is able to: unique names, a valid category, and a cascade so entries never outlive their exercise. Rules that can be settled from the input alone live in a validator. Rules that need a lookup — is this name taken, does this entry suit its exercise — live in the service layer.

Challenges Faced and How They Were Solved

1. Every rounded corner in the application was silently square

The problem. The stylesheet defined its corner radii as named values and referred to them by name, the way a web stylesheet would. Nothing rendered rounded. JavaFX resolves named values only for colours; used for a size, the value arrives as the wrong type, the conversion fails, and the declaration is discarded. The failure was silent in the interface and produced seventeen warnings on startup that were easy to overlook.

The fix. The thirty-eight affected declarations were given literal values, unchanged in effect, with a comment recording why they cannot be named. Startup is now free of warnings. The colour tokens were left alone, since colours are the one case JavaFX supports.

2. Changing an exercise’s category would have stranded its data

The problem. An exercise’s category decides which columns its entries fill. Switching an exercise from weightlifting to cardio after sessions had been logged would leave those sets and reps unreadable: the application would read them as a run with no distance and no time, displaying 0.00 mi in 00:00:00. No error would appear; the data would simply stop making sense.

The fix. The category is now fixed once an exercise has entries. The control is disabled with a reason given, rather than accepting the change and rejecting it on save, and the rule is enforced in the service layer so it holds no matter where the change comes from. Renaming is unaffected.

3. The progress chart quietly dropped sessions

The problem. Two sessions of the same exercise logged on one day appeared as a single point. The horizontal axis keeps one category per distinct label, and both sessions carried the same date as their label, so the second overwrote the first. Nothing failed; the chart was simply missing data — the worst kind of fault in a feature whose whole purpose is to show a complete history.

The fix. Repeated labels are now made distinct before plotting. The test written for this initially proved nothing, because it asserted on the chart’s stored data, which held both points either way. Asserting on the axis categories — where the merge actually happens — produced a test that fails when the fix is removed.

4. The interface tests failed at random

The problem. Tests that drive the interface passed roughly two runs in three. Failures were always a click that had not landed: a form field that never appeared because the button opening it was never pressed. The testing library moves the real mouse pointer at real screen coordinates, so any window taking focus mid-run swallowed a click. Waiting longer did not help, because the problem was the click itself.

The fix. The tests now render offscreen, so there is no window to lose focus and no pointer to intercept. That removes the cause rather than retrying around it, and has the side benefit that running the tests no longer takes over the display.

5. Text that fitted during development stopped fitting later

The problem. Several pieces of text were quietly cut off as the interface grew: buttons reading “Log wor…”, a date reading “Jul 26, 2…”, and a validation banner showing one and a half of its three messages. Each had a different immediate cause — a row with too much in it, a table column with a preferred width but no minimum, a dialog sized before its text was set — but all shared one property: nothing failed, so only looking at the screen revealed them.

The fix. Each was fixed at its cause: actions moved to their own row, every table column given a minimum width, and the banner made to grow to its content. Tests now assert that columns fit at the smallest window the application allows and that the banner is tall enough for its messages, so these cannot return unnoticed.

Testing

The project has 162 tests, run with `mvn test`. Most cover the database, validation and business rules directly. The remainder drive the real interface, clicking buttons and typing into forms, so that “every control works” is checked by the build rather than asserted by the author.

One practice was worth more than the count. Before trusting a new test, the behaviour it covered was deliberately broken to confirm the test failed. This caught two tests that passed whether or not the bug was present, including the one covering the chart fault described above. A test that cannot fail is worse than no test, because it is mistaken for coverage.

Future Improvements

- **Multiple profiles.** The database holds one person’s training. Adding a user table and a foreign key would let a household share an installation.
- **Notes on a session.** A free-text field for how a session felt, an injury, or a change of equipment — context the numbers do not carry.
- **Export.** Writing the history to CSV would let it be analysed elsewhere and would serve as a backup.
- **Goals.** Setting a target weight or pace and showing progress toward it would turn the chart from a record into something to aim at.
- **Comparing exercises.** The chart shows one exercise at a time; overlaying related lifts would show whether progress is general or specific.
- **A native installer.** Packaging with jpackage would remove the need for a JDK and Maven on the machine that runs it.

Compiling and Running

Requirements

- JDK 25 or newer.
- Apache Maven 3.9 or newer.

JavaFX does not need to be installed separately; Maven fetches it like any other dependency.

Commands

mvn clean javafx:run	Compiles and starts the application.
mvn clean package	Compiles and builds the jar without running.
mvn test	Runs all 162 tests, including the interface tests.

All three are run from the project directory, the one containing `pom.xml`. The database file `workout.db` is created there the first time the application starts, and its tables are created with it, so there is no setup step.

Appendix: Use of AI Tools

This project was built with Claude Code, an AI coding assistant, used throughout rather than for isolated snippets. Summarised rather than reproduced verbatim, the work divided as follows.

- **Planning.** Each stage was described in prose and agreed before code was written — the build order, the schema, the layering, and the breakdown of the interface into six reviewable pieces.
- **Implementation.** The assistant wrote the bulk of the code from those agreed plans, one stage at a time, with each stage run and checked before the next began.
- **Verification.** The assistant ran the build, the tests and the application itself after every change, and confirmed new tests could fail before trusting them.
- **Review.** Screenshots were taken at each stage and inspected. Several faults described in this report were found this way and would not have been caught otherwise.

Design decisions were made in discussion rather than delegated. The visual design language was supplied as a stylesheet and the interface built to it; the choice to chart pace rather than distance for cardio, and the layout of the progress view, came from review of working builds.